INGENIERÍA DE SISTEMAS COMPUTACIONALES

Técnicas de Programación Orientada a Objetos

SEMANA 4: COLECCIONES EN JAVA

Unidad I – Fundamentos de POO

Dr. Eric Gustavo Coronel Castillo
Ingeniería de Software | POO · 2026-1

Periodo Lectivo 2026-1

Java Collections

```
import java.util.*;

List<String> lista =
  new ArrayList<>();

lista.add("POO");
lista.add("Java");

Map<String,Integer> map = new HashMap<>();

Set<String> set = new HashSet<>();
```

REFLEXIÓN

Activando Saberes Previos

Reflexiona y responde antes de comenzar:

1

¿Qué sucede cuando necesitas almacenar una cantidad de datos desconocida en tiempo de compilación?

Imagina que estás desarrollando un sistema de matrícula. Los estudiantes pueden inscribirse en cualquier cantidad de cursos. ¿Cómo declararías las variables para almacenar esa información usando sólo arreglos simples? ¿Qué limitaciones encontrarías?

2

¿Cuál es la diferencia entre una lista, un conjunto y un mapa en términos de almacenamiento de datos?

Si tuvieras que representar: (a) una fila de espera de pacientes en un hospital, (b) los colores únicos disponibles en una tienda, y (c) el precio de cada producto identificado por su código, ¿qué tipo de estructura de datos utilizarías para cada caso? ¿Por qué?

RESPUESTAS

Fundamento Teórico

Respuesta 1: Limitaciones de los arreglos y necesidad de colecciones

Los arreglos en Java tienen tamaño fijo definido en tiempo de compilación. El Java Collections Framework (JCF) resuelve esto proporcionando estructuras de datos dinámicas. Según Bloch (2018), "las colecciones son preferibles a los arreglos porque son más flexibles y expresan mejor la intención del programador". El JCF fue introducido en Java 2 como un conjunto unificado de interfaces y clases para manejar grupos de objetos (Oracle, 2024).

Oracle. (2024). Java Collections Framework Overview. https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/doc-files/coll-overview.html

Respuesta 2: List, Set y Map – diferencias fundamentales

Según Vegas Gertrudix (2021), las tres interfaces principales del JCF tienen propósitos distintos: List (fila de pacientes) mantiene orden de inserción y permite duplicados; Set (colores únicos) no admite elementos duplicados y no garantiza orden; Map (precio por código) asocia claves únicas con valores. Esta diferenciación permite al programador seleccionar la estructura óptima según el problema, siguiendo el principio de adecuación semántica.

Vegas Gertrudix, J. M. (2021). Java 17: fundamentos prácticos de programación. eLibro. https://elibro.bibliotecaupn.elogim.com/es/lc/upnorte/titulos/222664

Estructura	Duplicados	Orden	Acceso por clave
List	✓ Sí	Inserción	No
Set	✗ No	No garantizado	No
Map	Clave única	Depende [illegible]	✓ Sí

DESARROLLO

¿Qué es el Java Collections Framework?

Definición: El Java Collections Framework (JCF) es una arquitectura unificada para representar y manipular colecciones de objetos. Proporciona interfaces, implementaciones y algoritmos estándar que permiten trabajar con grupos de datos de forma genérica, eficiente y reutilizable.

Interfaces

- Collection
- List
- Set
- Queue
- Map

Implementaciones

- ArrayList
- LinkedList
- HashSet
- TreeSet
- HashMap

Algoritmos

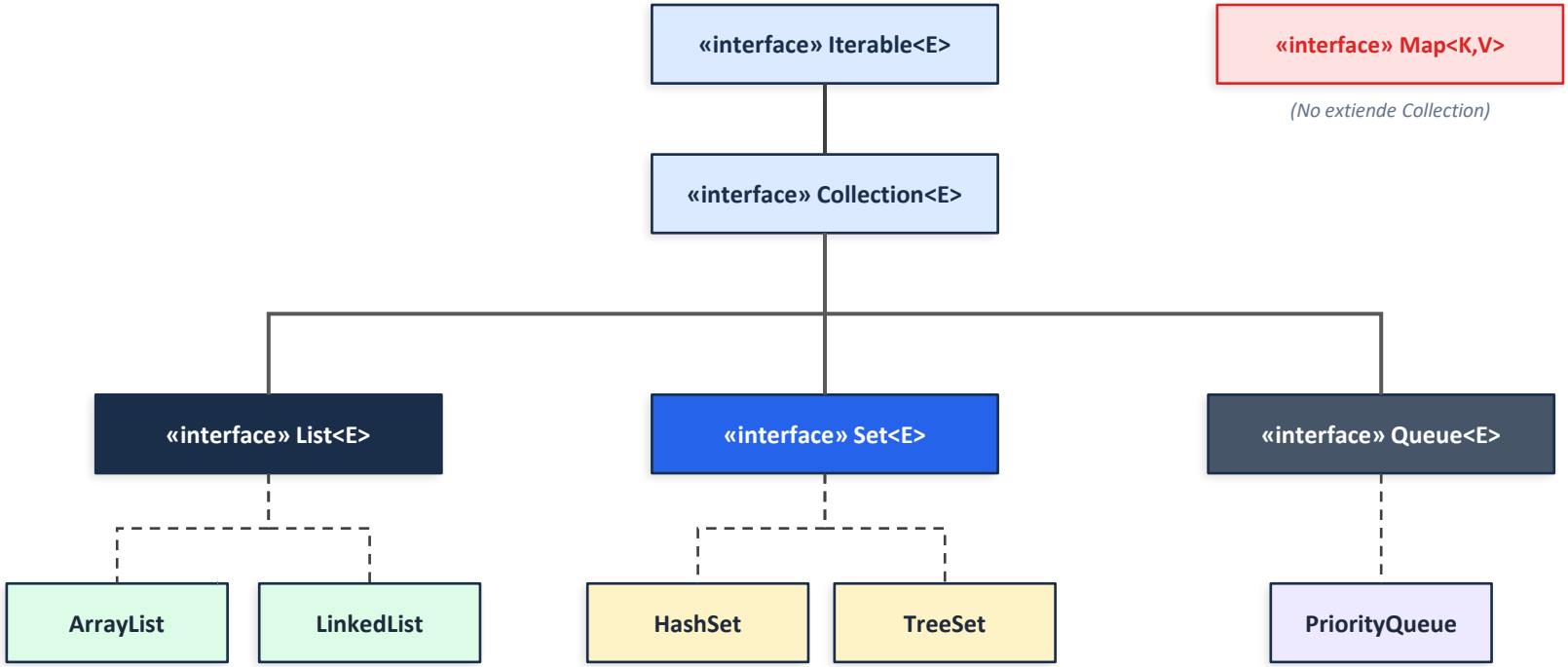
- Collections.sort()
- Collections.shuffle()
- Collections.min()
- Collections.max()
- Collections.reverse()

Oracle. (2024). Java SE 17 API Specification – java.util. https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/package-summary.html

Nota: Introducido en Java 2 (JDK 1.2, 1998). Extendido con genéricos en Java 5 (2004) para garantizar seguridad de tipos en tiempo de compilación.

DESARROLLO

Jerarquía del JCF – Interfaces Principales



Eckel, B. (2021). Thinking in Java (4th ed.). Prentice Hall. / Vegas Gertrudix, J. M. (2021). Java 17: fundamentos prácticos de programación.

DESARROLLO

Interface List – ArrayList y LinkedList

ArrayList<E>

- Implementa List usando un arreglo redimensionable dinámicamente.
- Acceso aleatorio O(1) por índice.
- Inserción/eliminación al final O(1) amortizado; en medio O(n).
- No sincronizado (no thread-safe por defecto).
- Permite elementos null.
- Capacidad inicial: 10 elementos (Java 17).

LinkedList<E>

- Implementa List y Deque usando lista doblemente enlazada.
- Inserción/eliminación O(1) en cualquier posición (conociendo el nodo).
- Acceso aleatorio O(n) – recorre nodos.
- Mayor consumo de memoria (overhead por nodo).
- Útil como pila (Stack) o cola (Queue).

Ejemplo: ArrayList en Java

```java
import java.util.ArrayList;
import java.util.List;

List<String> cursos = new ArrayList<>();

// Agregar elementos
cursos.add("POO");
cursos.add("Estructuras");
cursos.add(1, "Algoritmos"); // en pos 1

// Recorrer con for-each
for (String c : cursos) {
  System.out.println(c);
}

// Eliminar y verificar
cursos.remove("POO");
System.out.println(cursos.size());// 2
```

Vegas Gertrudix, J. M. (2021). Java 17: fundamentos prácticos de programación. https://elibro.bibliotecaupn.elogim.com/es/lc/upnorte/titulos/222664

DESARROLLO

Interface Set – HashSet y TreeSet

HashSet<E>

- No permite duplicados.
- No garantiza orden de iteración.
- Operaciones add, remove, contains: O(1) promedio.
- Permite un elemento null.
- Basado en tabla hash (HashMap internamente).

TreeSet<E>

- No permite duplicados.
- Mantiene orden natural o por Comparator.
- Operaciones: O(log n) – árbol rojo-negro.
- No permite elementos null.
- Implementa NavigableSet (first, last, subSet).

LinkedHashSet<E>

- No permite duplicados.
- Mantiene orden de inserción.
- Rendimiento similar a HashSet.
- Útil cuando se necesita iteración predecible.
- Combina tabla hash + lista enlazada.

Ejemplo práctico – HashSet:

```
Set<String> colores = new HashSet<>();  colores.add("rojo"); colores.add("azul"); colores.add("rojo"); // duplicado ignorado
System.out.println(colores.size()); // 2    System.out.println(colores.contains("azul")); // true
```

Oracle. (2024). java.util.Set – Java SE 17 API. https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Set.html

DESARROLLO

Interface Map – HashMap y TreeMap

Map<K,V>: Estructura que asocia claves únicas (K) con valores (V). No extiende Collection. Cada clave mapea exactamente un valor. Las claves no pueden repetirse; si se inserta la misma clave, el valor se reemplaza.

HashMap<K,V>

- No garantiza orden de claves.
- Operaciones get, put, remove: O(1) promedio.
- Permite una clave null y múltiples valores null.
- No sincronizado.
- Implementación más usada.

TreeMap<K,V>

- Mantiene claves en orden natural o Comparator.
- Operaciones: O(log n) – árbol rojo-negro.
- No permite clave null.
- Implementa NavigableMap.
- Útil para rangos de claves (subMap, headMap).

Ejemplo práctico – HashMap:

```
Map<String, Double> precios = new HashMap<>();
precios.put("COMP001", 45.50);  precios.put("COMP002", 89.99);
precios.put("COMP001", 42.00); // reemplaza el valor anterior
System.out.println(precios.get("COMP001")); // 42.0
for (Map.Entry<String,Double> e : precios.entrySet()) {  System.out.println(e.getKey()+": "+e.getValue()); }
```

Oracle. (2024). java.util.Map – Java SE 17 API. https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Map.html

DESARROLLO

Comparación de Estructuras – ¿Cuándo usar cada una?

Estructura	Duplicados	Orden	Acceso	Caso de uso típico
ArrayList	✓ Sí	Inserción	O(1) por índice	Lista de ítems, historial de navegación
LinkedList	✓ Sí	Inserción	O(n) secuencial	Cola de mensajes, pila de operaciones
HashSet	✗ No	Sin orden	O(1) promedio	Eliminar duplicados, caché de IDs
TreeSet	✗ No	Natural/Custom	O(log n)	Ranking, rangos ordenados
HashMap	Claves únicas	Sin orden	O(1) por clave	Diccionario, catálogo de productos
TreeMap	Claves únicas	Natural/Custom	O(log n)	Agenda telefónica, índice de libro

Regla de oro (Bloch, 2018): Programa contra la interface, no la implementación. Declara tus variables como `List<T>`, `Set<T>` o `Map<K,V>`; instancia con la clase concreta. Esto facilita el cambio de implementación sin modificar el contrato.

Bloch, J. (2018). Effective Java (3rd ed., Ítem 64). Addison-Wesley. / Vegas Gertrudix, J. M. (2021). Java 17: fundamentos prácticos de programación.

DESARROLLO

Iteración y Clase java.util.Collections

Formas de iterar una colección

```
List<Integer> nums =
  Arrays.asList(3,1,4,1,5,9);

// 1. For-each (recomendado)
for (int n : nums)
  System.out.print(n + " ");

// 2. Iterator explícito
Iterator<Integer> it = nums.iterator();
while (it.hasNext())
  System.out.print(it.next());

// 3. Stream (Java 8+)
nums.stream()
  .filter(n -> n > 3)
  .forEach(System.out::println);

// 4. forEach con lambda
nums.forEach(n -> System.out.print(n));
```

Clase java.util.Collections

`sort(list)`	Ordena ascendentemente (Comparable)
`sort(list, comp)`	Ordena con Comparator personalizado
`reverse(list)`	Invierte el orden de los elementos
`shuffle(list)`	Mezcla aleatoriamente los elementos
`min(collection)`	Retorna el elemento mínimo
`max(collection)`	Retorna el elemento máximo
`frequency(c, o)`	Cuenta ocurrencias de o en c
`unmodifiableList(l)`	Crea vista inmutable de la lista

Oracle. (2024). java.util.Collections. https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Collections.html

DESARROLLO

Genéricos (Generics) en Colecciones

¿Por qué usar genéricos? Sin genéricos, las colecciones almacenan `Object` y requieren castings manuales. Los genéricos (introducidos en Java 5) garantizan ***type safety*** en tiempo de compilación, eliminando `ClassCastException` en tiempo de ejecución (Bloch, 2018, Ítem 26-29).

❌ Sin Genéricos (antes de Java 5)

```
List lista = new ArrayList();
lista.add("Hola");
lista.add(42); // compila!

// Requiere casting manual:
String s = (String) lista.get(0); // OK
String x = (String) lista.get(1);
// ClassCastException en runtime!
```

✓ Con Genéricos (Java 5+)

```
List<String> lista = new ArrayList<>();
lista.add("Hola");
// lista.add(42); // ERROR de compilación

// Sin casting:
String s = lista.get(0); // seguro

// Tipo verificado en compilación.
```

Bloch, J. (2018). Effective Java (3rd ed., Items 26–29). Addison-Wesley. / Vegas Gertrudix, J. M. (2021). Java 17: fundamentos prácticos de programación.

ACTIVIDAD EN CLASE

Taller de Análisis de Problemas con Colecciones

Aplica los conceptos de List, Set y Map para resolver problemas reales.

Ejercicio 1

Gestión de Matrícula

Ejercicio 2

Inventario de Productos

Ejercicio 3

Directorio Telefónico

Ejercicio 4

Análisis de Frecuencias

Tiempo estimado: 35–40 minutos

ACTIVIDAD 1

Gestión de Matrícula Universitaria

Contexto: El sistema de matrícula de la universidad necesita gestionar la lista de cursos en los que se inscribe un estudiante durante el semestre. Los cursos deben poder agregarse, eliminarse y consultarse.

Requerimientos a implementar:

a) Crear una lista de cursos matriculados usando ArrayList<String>.
b) Agregar al menos 5 cursos (incluyendo intentos de duplicados).
c) Mostrar la lista completa de cursos usando for-each.
d) Eliminar un curso por nombre y verificar si un curso existe con contains().
e) Mostrar cuántos cursos tiene el estudiante con size().

- **f) RETO: Ordenar la lista alfabéticamente usando Collections.sort() y mostrar el resultado.**

```
// Salida esperada (f):  [Algoritmos, Cálculo, Estructuras, Java POO, Redes]
```

ACTIVIDAD 2

Inventario de Productos con Set y Map

Contexto: Una ferretería necesita un sistema básico de inventario. Debe controlar categorías únicas de productos y el precio de cada producto identificado por su código.

Parte A: Categorías con HashSet

- Crear HashSet<String> de categorías.
- Agregar: Herramientas, Pinturas, Eléctricos, Herramientas (duplicado).
- Mostrar el Set resultante.
- Verificar si 'Pinturas' existe.
- Eliminar 'Eléctricos' y mostrar tamaño final.

Parte B: Precios con HashMap

- Crear HashMap<String,Double> de productos.
- Agregar pares: F001-15.50, F002-89.99, F003-7.25.
- Consultar precio de F002.
- Actualizar precio de F001 a 12.00.
- Iterar e imprimir todos los productos con entrySet().

🏆 **RETO:** Usando TreeMap, muestra los productos ordenados alfabéticamente por código.

ACTIVIDAD 3

Directorio Telefónico y Análisis de Frecuencias

Ejercicio 3A: Directorio Telefónico

- Crear HashMap<String,String> (nombre → teléfono).
- Agregar 5 contactos.
- Buscar teléfono por nombre.
- Mostrar si un contacto existe (containsKey).
- Listar todos los contactos (keySet).
- **RETO: Usar TreeMap para listarlos en orden alfabético.**

Ejercicio 3B: Análisis de Frecuencias

Dada la lista:

```
["Java","POO","Java","C++","POO","Java","Python"]
```

- Usar HashMap<String,Integer> para contar frecuencia de cada palabra.
- Mostrar el resultado: Java=3, POO=2, C++=1, Python=1.
- Identificar la palabra más repetida.
- **RETO: Usar Collections.frequency() para verificar los conteos.**

💡 Tips para el desarrollo

```
// Conteo de frecuencias con getOrDefault:
freq.put(palabra, freq.getOrDefault(palabra, 0) + 1);

// Verificar existencia antes de acceder:
if (directorio.containsKey(nombre)) { ... }

// Para TreeMap con orden inverso:
new TreeMap<>(Comparator.reverseOrder());
```

CONCLUSIONES

Lo que aprendimos hoy

01 El JCF es esencial en Java moderno

El Java Collections Framework provee un conjunto unificado y probado de interfaces e implementaciones. Dominarlas reduce código y errores, y es requisito para cualquier desarrollo profesional.

02 List, Set y Map tienen propósitos distintos

List para secuencias con duplicados, Set para conjuntos sin duplicados, Map para asociaciones clave-valor. Elegir la estructura correcta determina la eficiencia y claridad del código.

03 Los genéricos garantizan type safety

Programar con genéricos (List<T>, Map<K,V>) elimina ClassCastException en tiempo de ejecución y hace el código más legible y mantenible (Bloch, 2018).

04 Programa contra la interfaz

Declara variables como List, Set o Map; no como ArrayList o HashMap. Este principio facilita el cambio de implementación sin modificar el contrato del código (SOLID – OCP).

¿Preguntas?

Este es el momento para resolver tus dudas sobre:

- ¿Cuándo usar ArrayList vs LinkedList?
- ¿Cómo funciona internamente HashMap?
- ¿Cuál es la diferencia entre Iterator y for-each?
- ¿Cómo implementar Comparable y Comparator?
- ¿Qué son las colecciones inmutables?

¡Toda pregunta es válida y enriquece el aprendizaje!

REFERENCIAS BIBLIOGRÁFICAS

Normas APA – 7.ª edición

Básica

Vegas Gertrudix, J. M. (2021). Java 17: fundamentos prácticos de programación. eLibro.
https://elibro.bibliotecaupn.elogim.com/es/lc/upnorte/titulos/222664

Complementaria

Blasco, F. (2020). Programación en Java. Digitalia.
https://digitalia.bibliotecaupn.elogim.com/a/110219

Jiménez de Parga, C. (2021). UML: arquitectura de aplicaciones en Java, C++ y Python. eLibro.
https://elibro.bibliotecaupn.elogim.com/es/lc/upnorte/titulos/222720

Blasco, F. (2020). Programación Java: JDBC y Swing. eLibro.
https://elibro.bibliotecaupn.elogim.com/es/lc/upnorte/titulos/222696

Fuentes Técnicas

Oracle. (2024). Java Collections Framework Overview. Java SE 17 Documentation.
https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/doc-files/coll-overview.html

Oracle. (2024). java.util – Package Summary. Java SE 17 API.
https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/package-summary.html

Bloch, J. (2018). Effective Java (3rd ed.). Addison-Wesley.